HACKATHON PROJECT CONCEPT | SOFTWARE ENGINEERING + AI

REGRESSION RADAR

Predict risky code changes before they reach production.

THE IDEA
A pull-request risk reviewer that analyses code changes and a repository's Git history to flag patterns associated with past regressions—then recommends what to inspect before merging.

THE PROMISE
Not "AI says this code is bad." Instead: evidence-based risk signals, explainable scoring and practical next actions at review time.

1. The problem, in plain English

When developers change one part of an application, they can unintentionally break another part that used to work. This is called a **regression**. A pull request (PR) is the proposed code change that a team reviews before merging it into the main codebase. Reviewers can inspect the code, but they may not know which files have repeatedly caused bugs, which files usually change together, or which tests are most relevant.

Example
A PR changes `payments.ts`. In the repository's history, payment changes often appeared alongside `payments.test.ts` and `refunds.ts`. This PR changes only the payment file. That does not prove a bug exists—but it is a useful review signal.

2. What Regression Radar checks

SIGNAL	WHAT IT LEARNS	WHAT IT FLAGS
Historical hotspots	Which files or functions were frequently involved in bug-fix or revert commits.	A PR touches a historically fragile area; reviewers should inspect it carefully.
Co-change patterns	Which files commonly changed together across previous commits.	A related file or test that often changes with this file is absent from the PR.
Ownership signal	Who has recently changed or reviewed the affected modules, where repository data supports it.	Suggests a knowledgeable reviewer. It cannot infer leave or availability from Git history alone.
Test-gap hints	Links changed files to nearby test files and historical test patterns.	Recommends a short list of tests to run; it does not claim those tests guarantee safety.

Core distinction: Regression Radar is not a style checker or a general code-quality score. It is designed to provide **PR-time, history-aware risk signals and recommended actions**.

From Git history to a useful PR review

Keep the first version focused: analyse a repository, score a proposed change, and show the evidence.

3. How the system works

1	**Read the pull request** Get the changed files and diff from GitHub—or provide a local diff for the CLI demo.
2	**Analyse repository history** Read commits, bug-fix/revert labels where identifiable, file co-change patterns and contributor activity.
3	**Calculate a transparent risk score** Combine weighted signals such as hotspot history, missing co-changed files and test gaps. Keep the scoring deterministic.
4	**Recommend next actions** Suggest relevant tests and reviewers. Use an LLM only to turn the evidence into a concise explanation—not to invent the score.
5	**Show the result in the PR** Post a comment with the score, reasons, evidence and recommended checks before merge.

4. What the developer sees

REGRESSION RADAR · PR #42

Risk score: 72 / 100 — HIGH

Illustrative score only; not a 72% probability of failure.

Why it was flagged

- Payment module has repeated bug-fix history.
- A commonly co-changed test file is absent.
- Refund logic is historically related to this area.

Suggested checks

1. Run payment integration tests.
2. Run refund and order-processing tests.
3. Request review from a recent payment-module contributor.

5. Four-person build plan

PERSON	OWNERSHIP	FIRST DELIVERABLE
1 · Git analytics	Git history parser and hotspot scoring	File-level history metrics from a local repository
2 · PR integration	GitHub API / GitHub Actions and PR comment	Fetch changed files and publish a report
3 · Pattern engine	Co-change analysis, test-gap hints and scoring	Explainable flags with evidence
4 · Validation & demo	Replay dataset, ownership hints, testing and documentation	A reproducible demo with measured results

6. Build order and honest evaluation

BUILD IN THIS ORDER

1. Local CLI that reads Git history and scores a diff.
2. Hotspot + co-change + test-gap signals.
3. GitHub Action that comments on a PR.
4. Optional LLM explanation and reviewer suggestion.

PROVE IT WITH A REPLAY

Choose a public repository with identifiable bug-fix or revert commits. Reconstruct the pre-fix change, score it using only information available at that time, and report detection rate and false-positive rate.

Important limitations: Git history is an imperfect signal. A file touched by past bugs is not necessarily unsafe, and an unchanged test file is not automatically a missing test. The score is a prioritisation aid—not a guarantee, a production-failure probability, or a replacement for code review and CI tests.